

The Language of Modern AI — A Living Lexicon



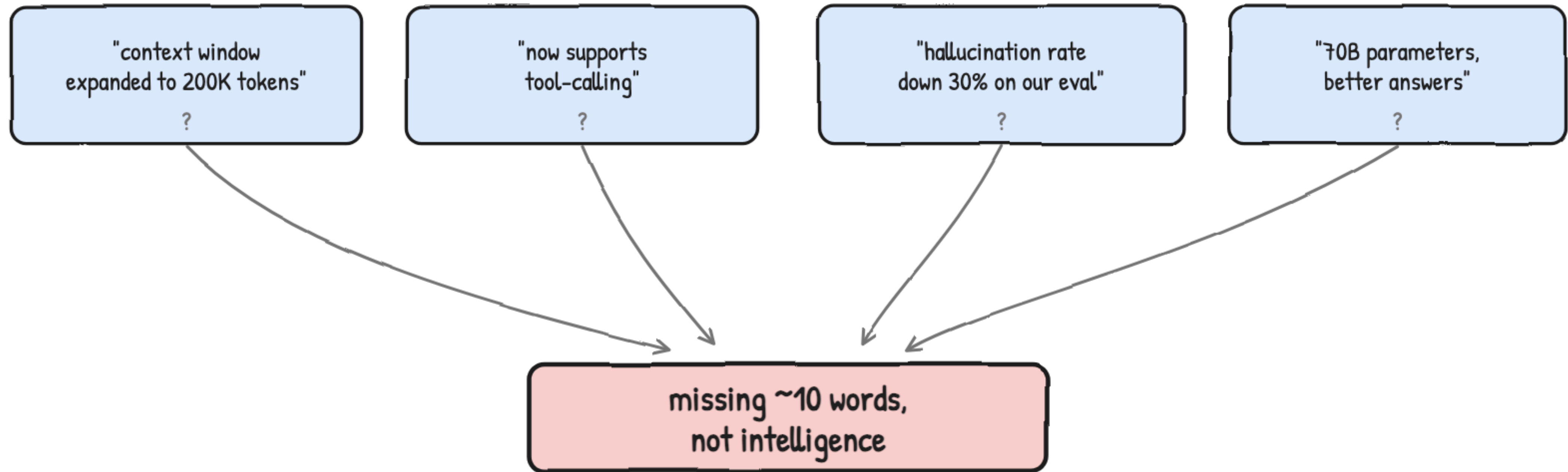
Ten words turn release notes from noise into information

Gourav Shah · School of DevOps & AI · Lesson + Lab + Quiz



Why does every AI product page sound like a different language?

Every AI release note is built from ten words you don't own yet



Plain English, wired together to mean nothing — yet



The phrasebook effect

You don't need fluency in a country — you need thirty words



Numbers, directions, a menu item or two

Ten precise words are your AI phrasebook

30 words

get through a country

numbers

directions

a menu item or two

10 words

follow any AI pitch

tokens · parameters · embeddings

context window · RAG

agent · tool-calling

eval · guardrail · hallucination

Not fluency — just the right ten words



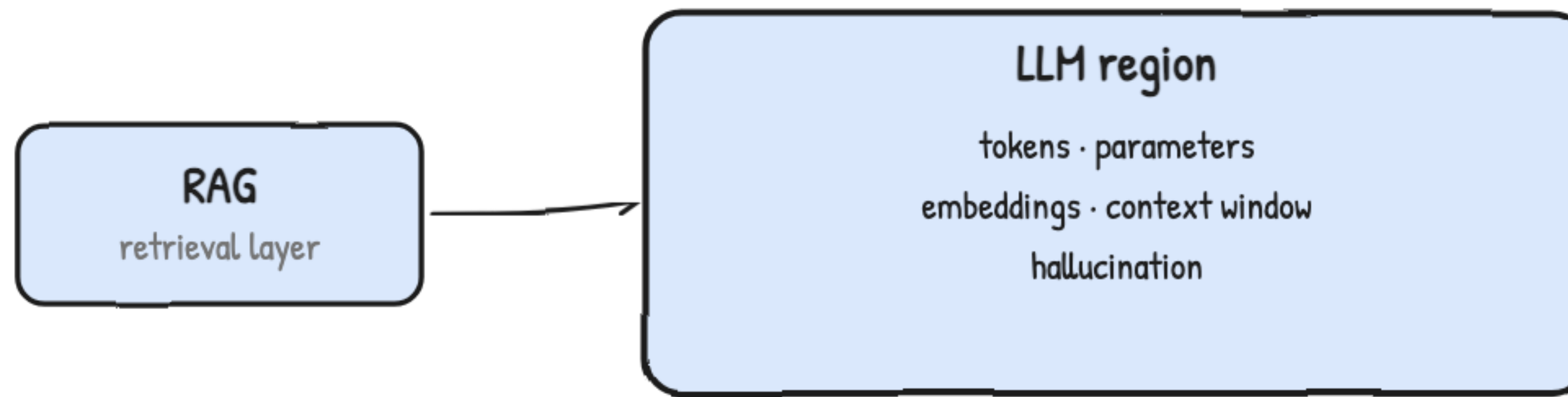
Where these words live on the map

Every word has an address on the AI Map



Where each of the ten terms lives on the map

Every word has an address on the AI Map



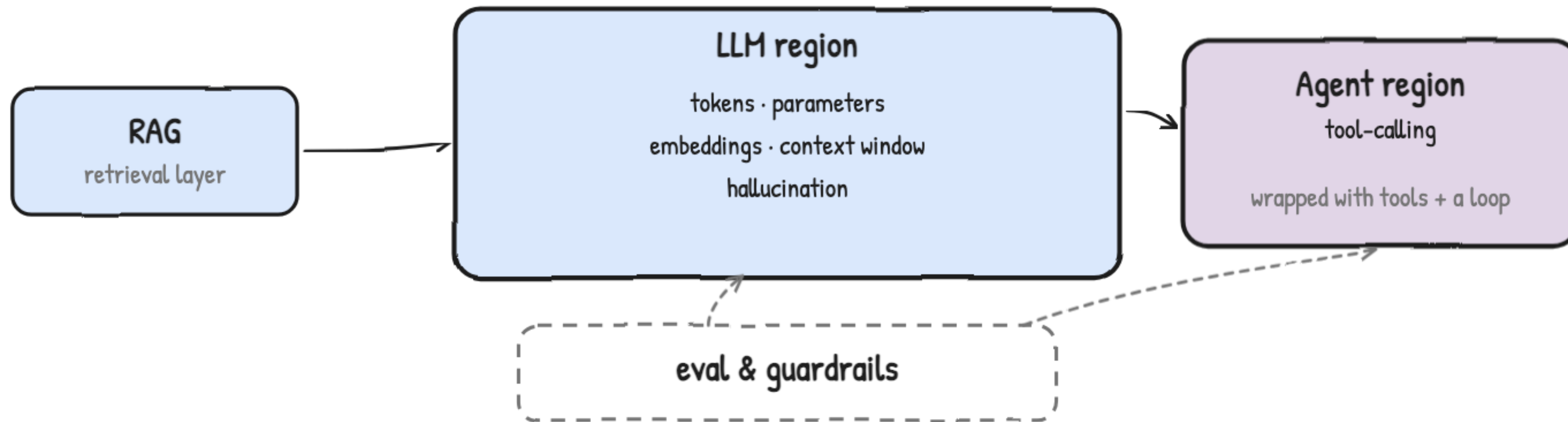
Where each of the ten terms lives on the map

Every word has an address on the AI Map

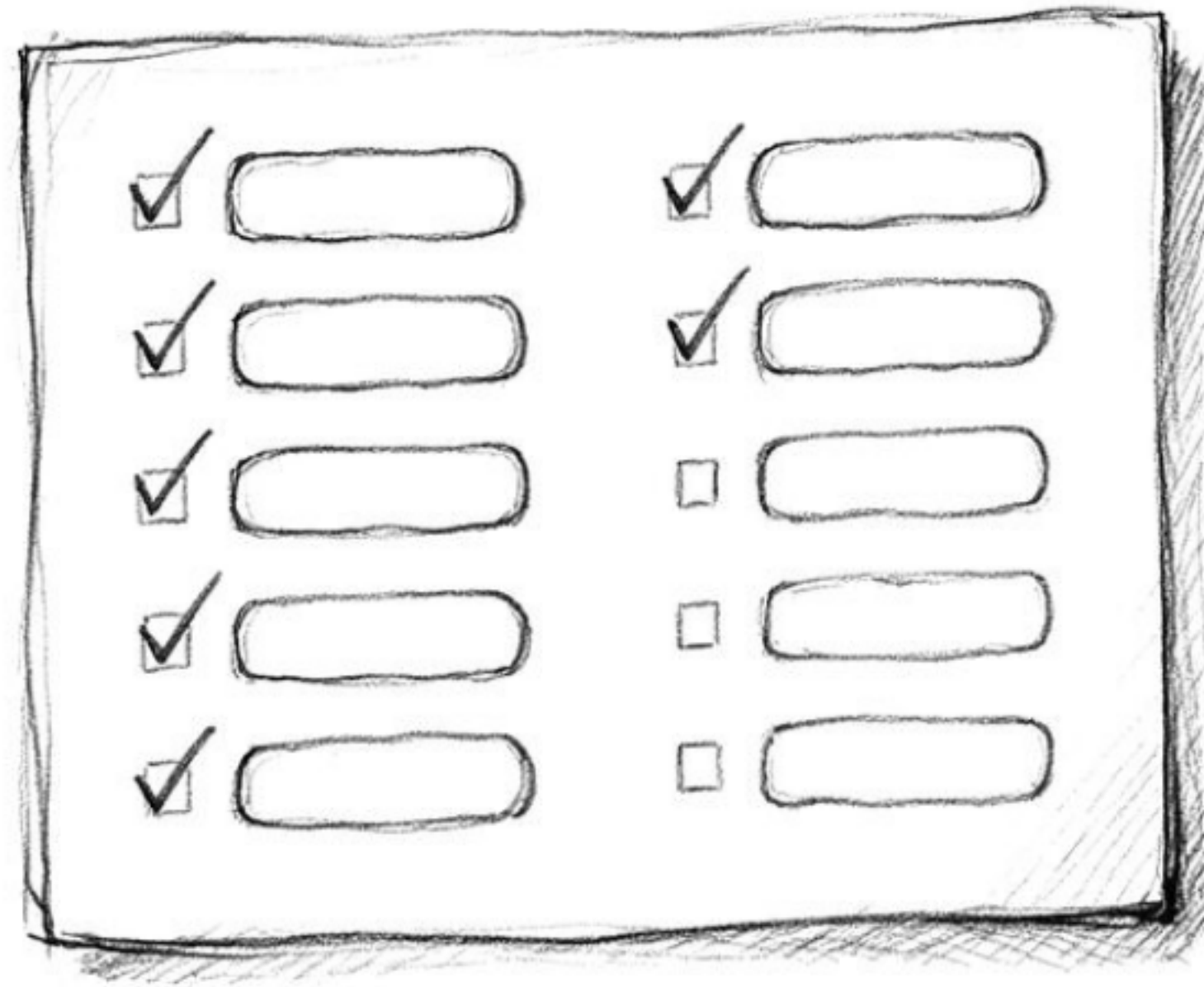


Where each of the ten terms lives on the map

Every word has an address on the AI Map

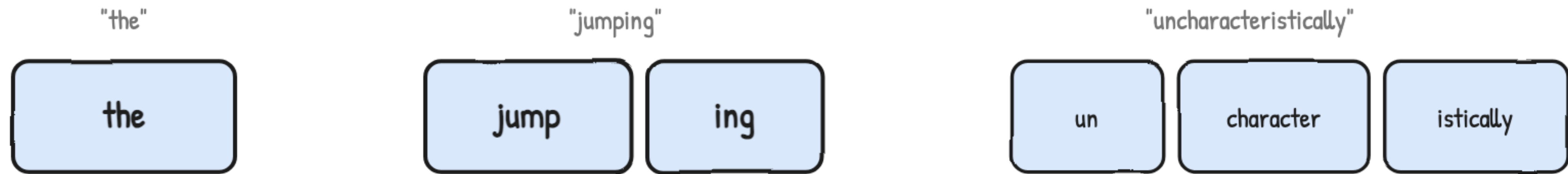


Where each of the ten terms lives on the map



The ten terms

A token is the chunk the model reads, writes, and bills

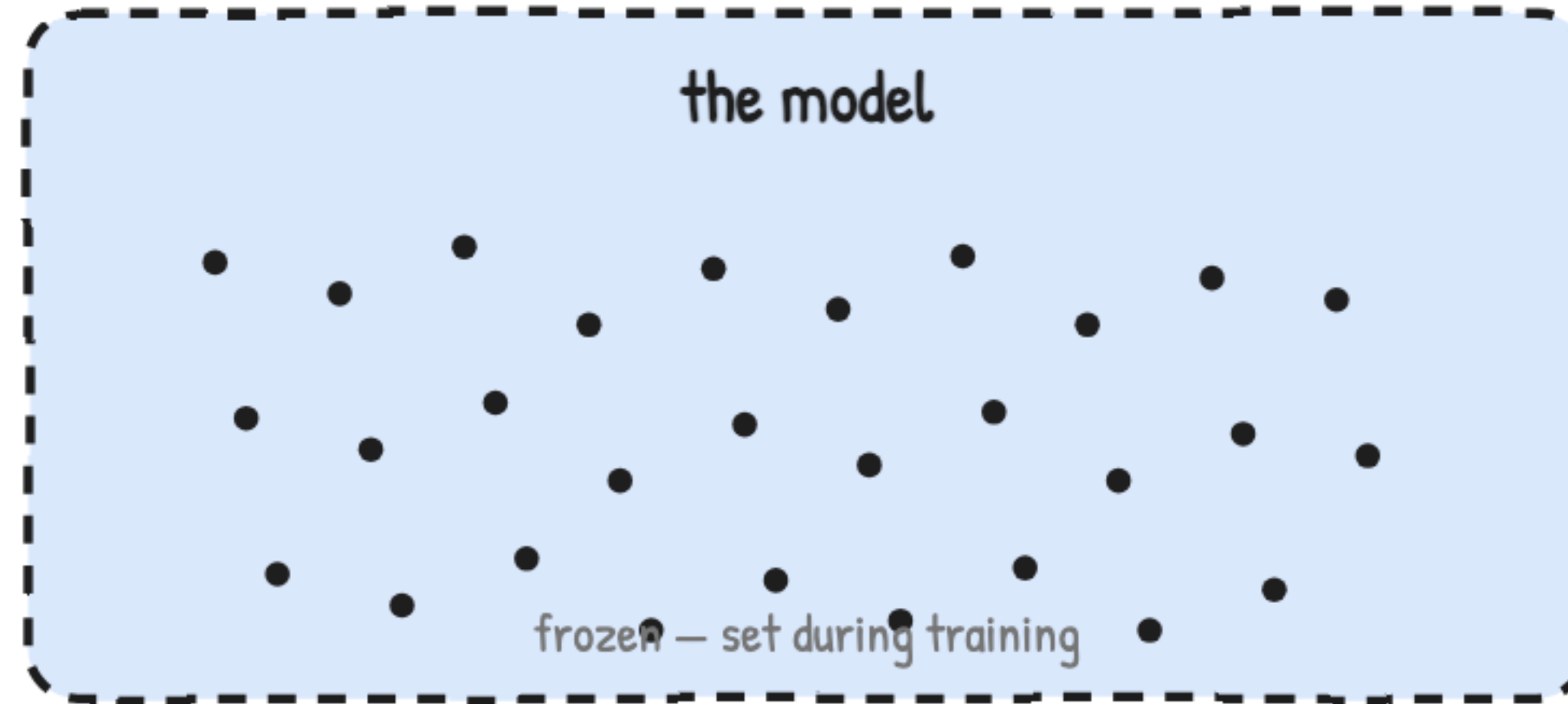


2.3 million tokens processed this month

= a bill, not a capability

"jumping" → jump + ing — two tokens, not one word

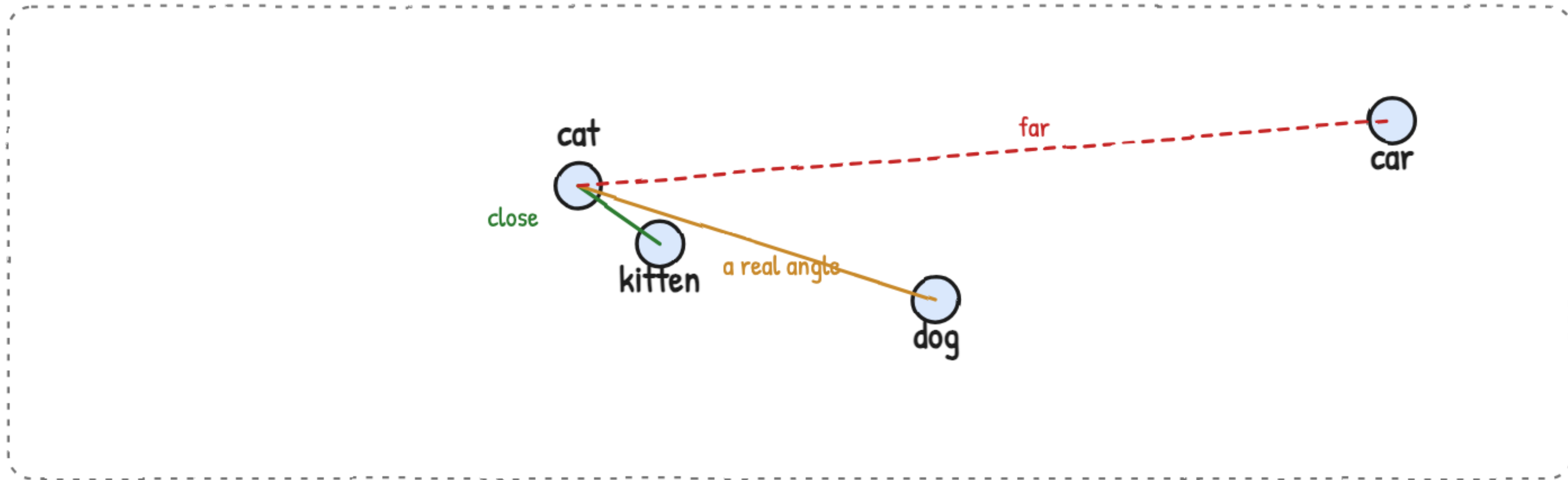
Parameters are the numbers a model learned, frozen at training



70,000,000,000

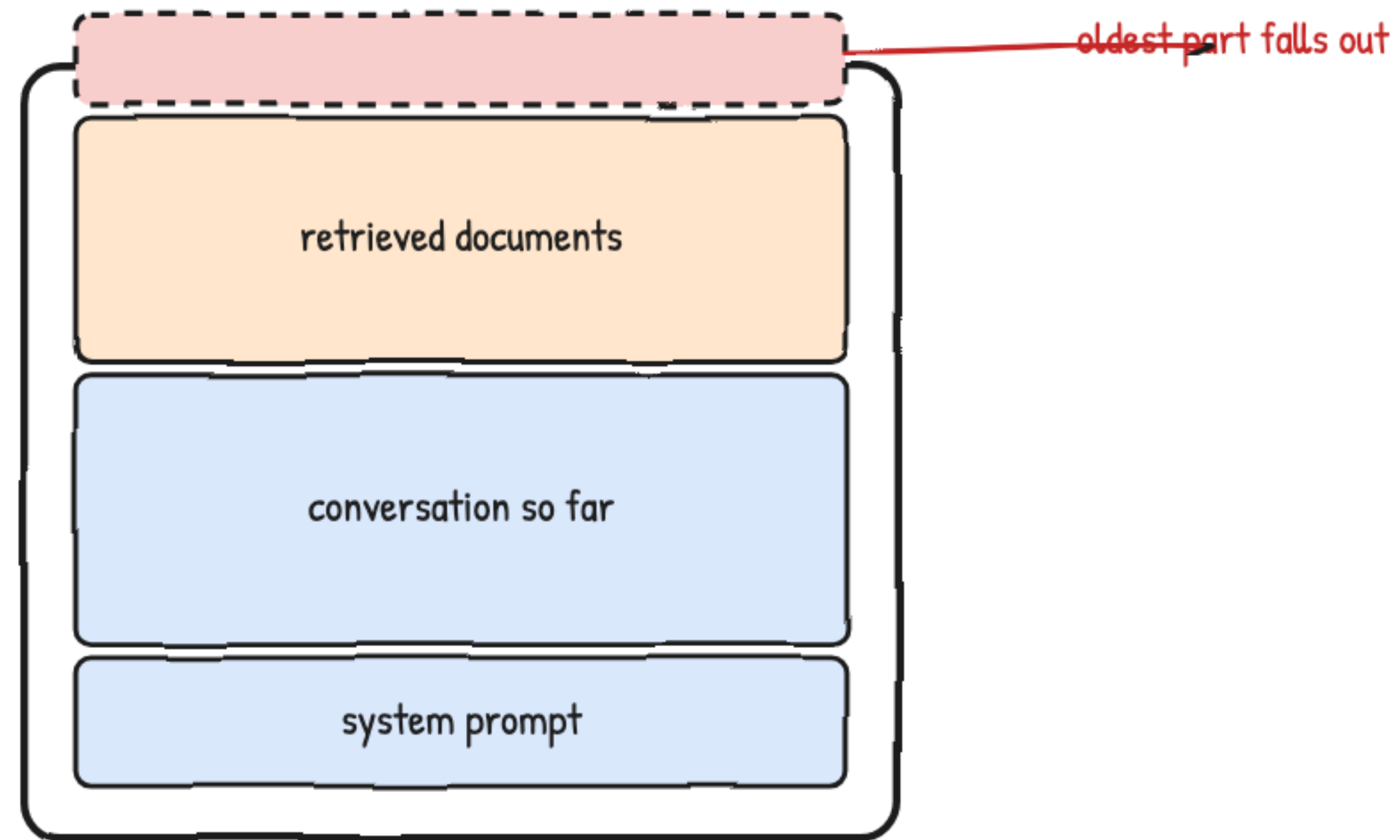
70 billion of them, baked in, untouched at run-time

Embeddings turn meaning into a distance you can calculate



Close together = similar, far apart = unrelated

The context window is a hard limit, not a memory



Prompt + history + retrieved text, all counted together

RAG retrieves first, then writes the answer grounded in it



your
documents

and if retrieval finds nothing?

Retrieve → generate — a two-step pipeline

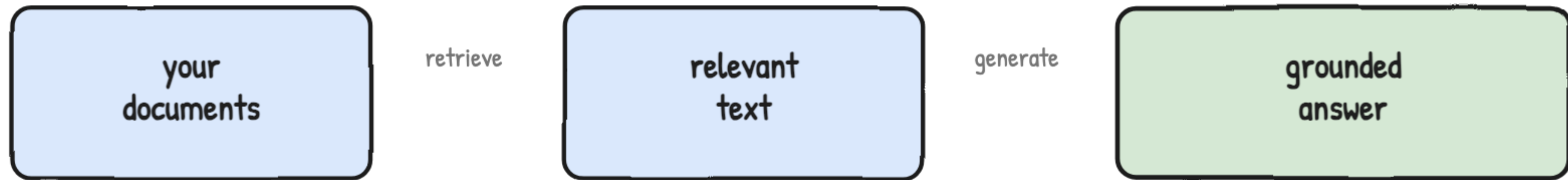
RAG retrieves first, then writes the answer grounded in it



and if retrieval finds nothing?

Retrieve → generate — a two-step pipeline

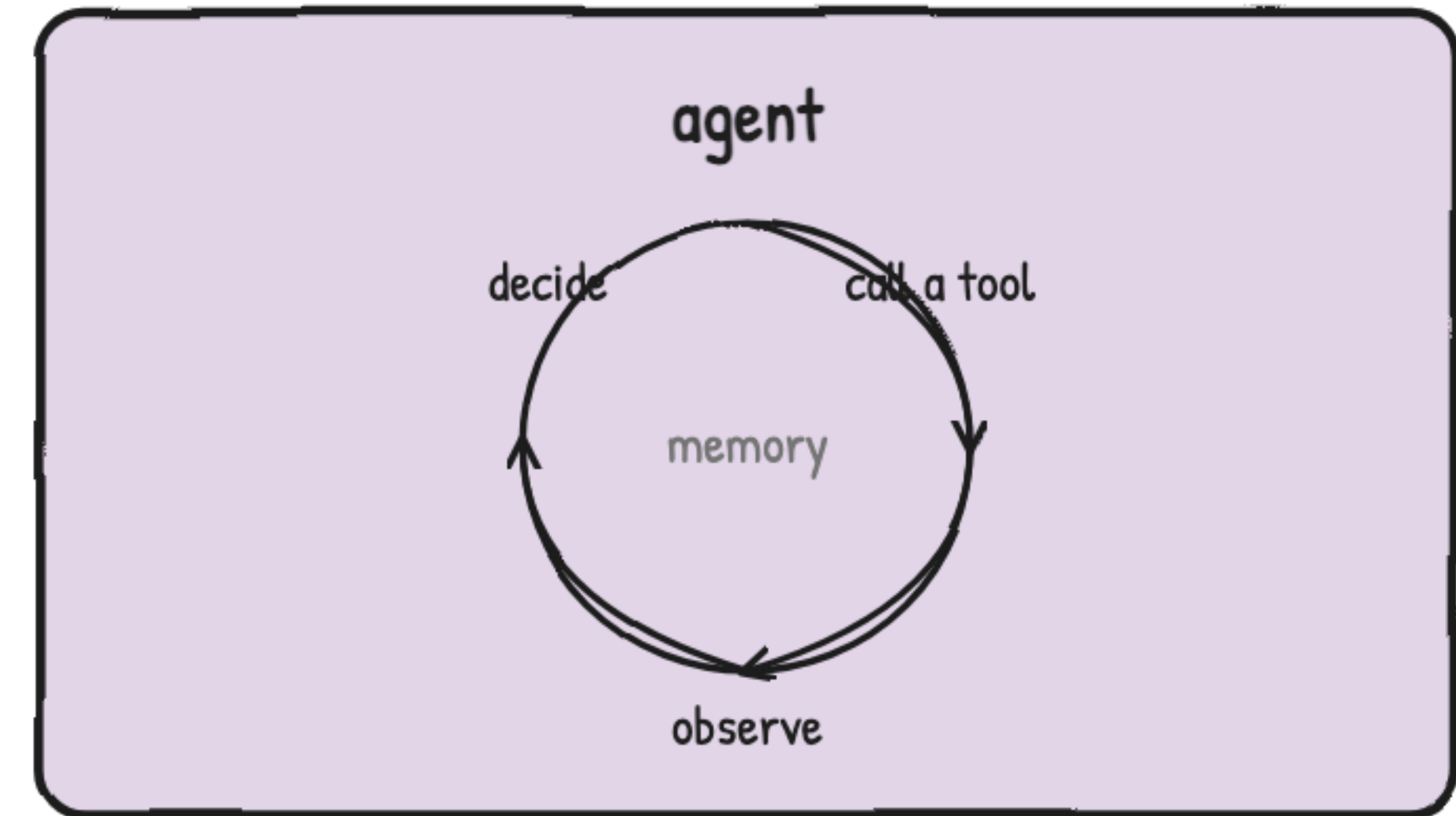
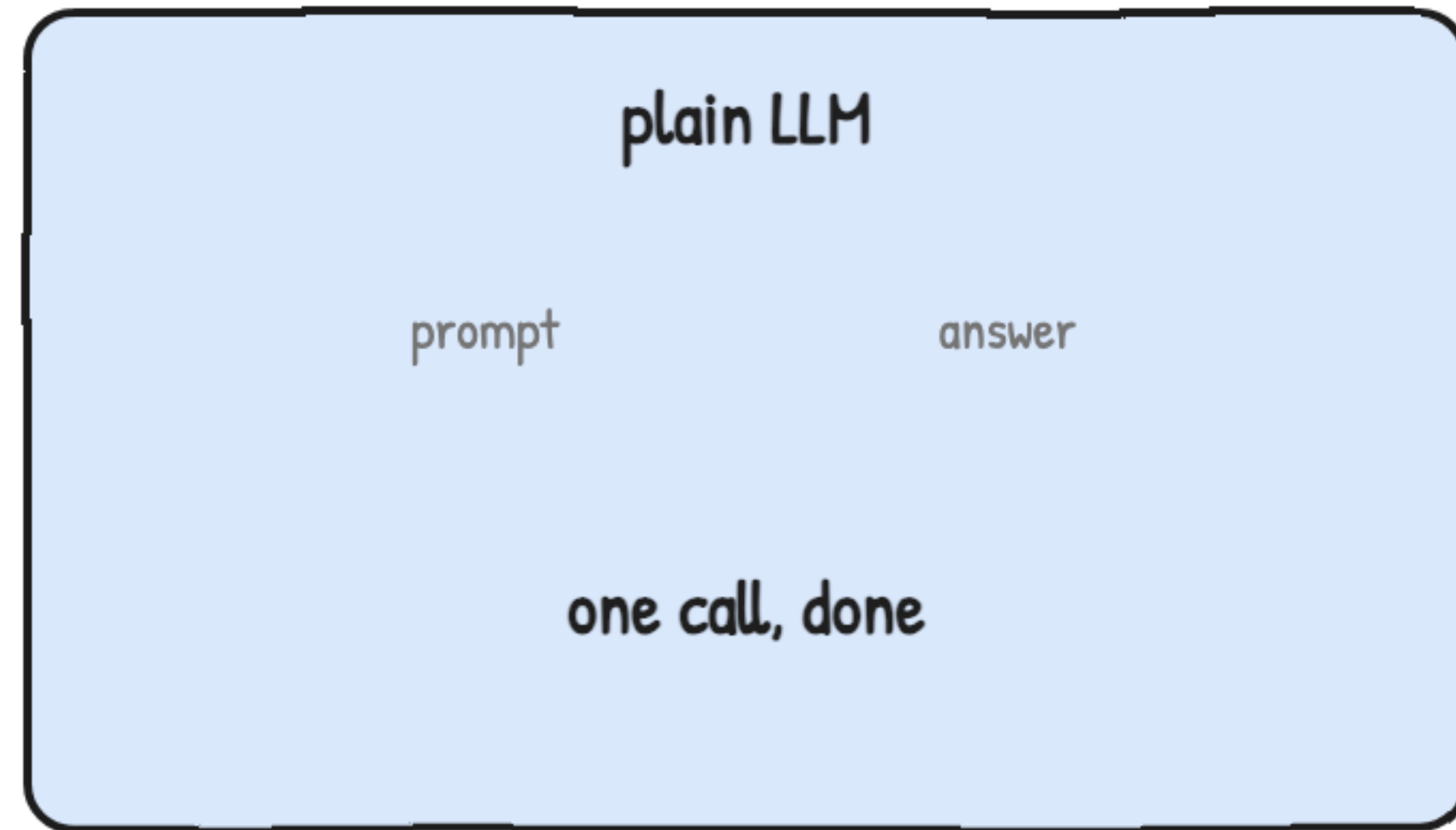
RAG retrieves first, then writes the answer grounded in it



and if retrieval finds nothing?

Retrieve → generate — a two-step pipeline

An agent loops with tools; a plain LLM answers once and stops



The loop is what makes it an agent, not a persona

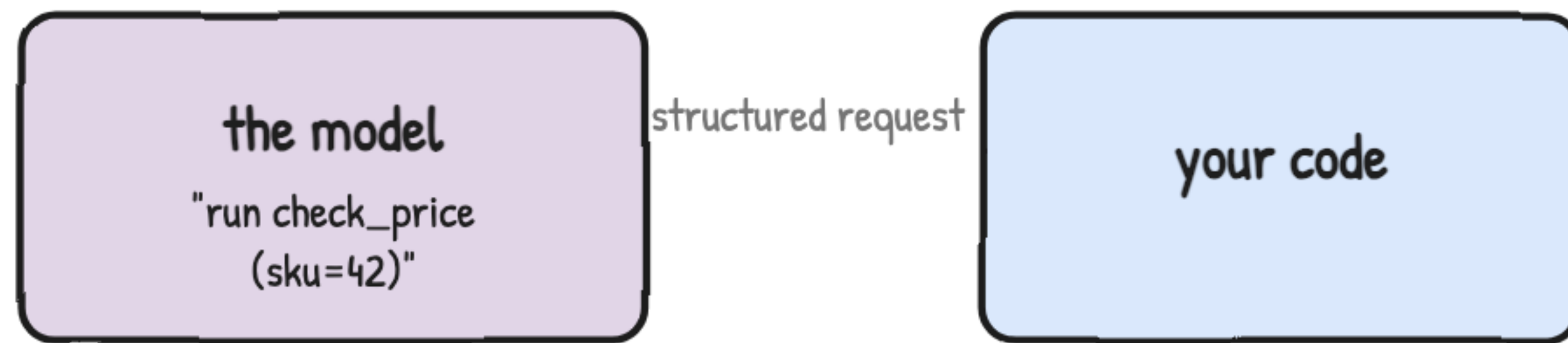
Tool-calling is the wire signal that lets an agent act

the model

```
"run check_price  
(sku=42)"
```

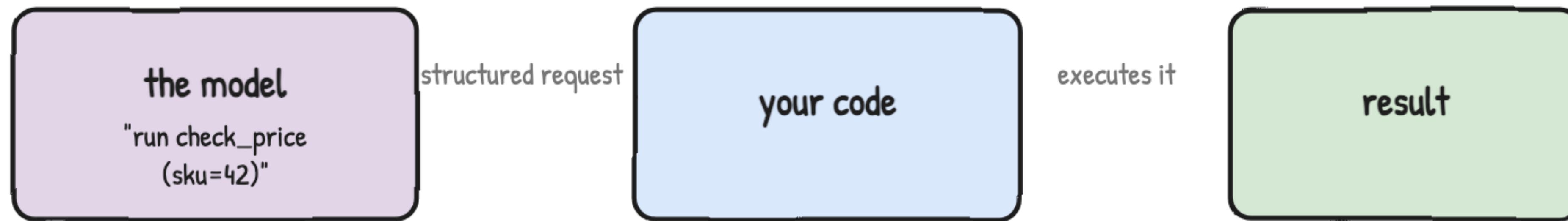
A structured request, not a direct answer

Tool-calling is the wire signal that lets an agent act



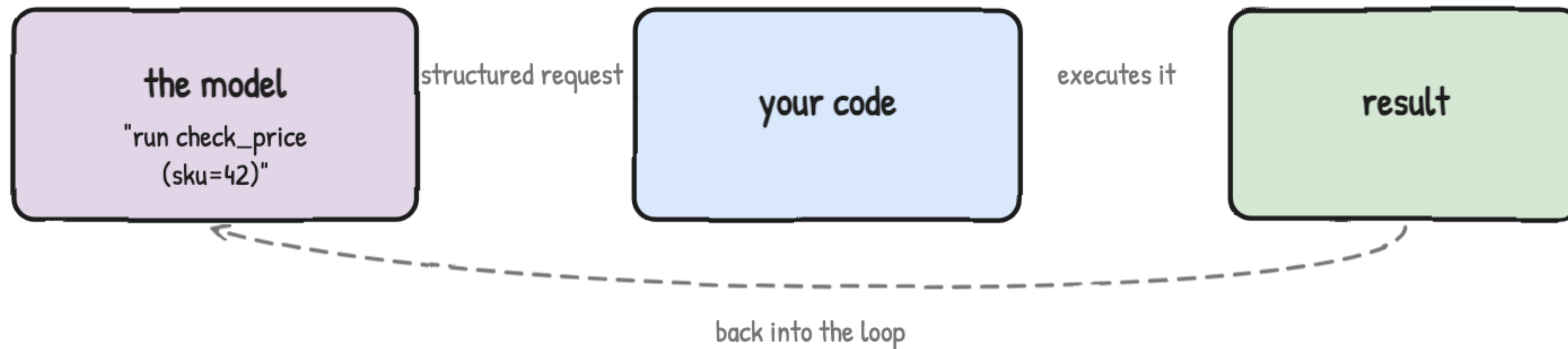
A structured request, not a direct answer

Tool-calling is the wire signal that lets an agent act



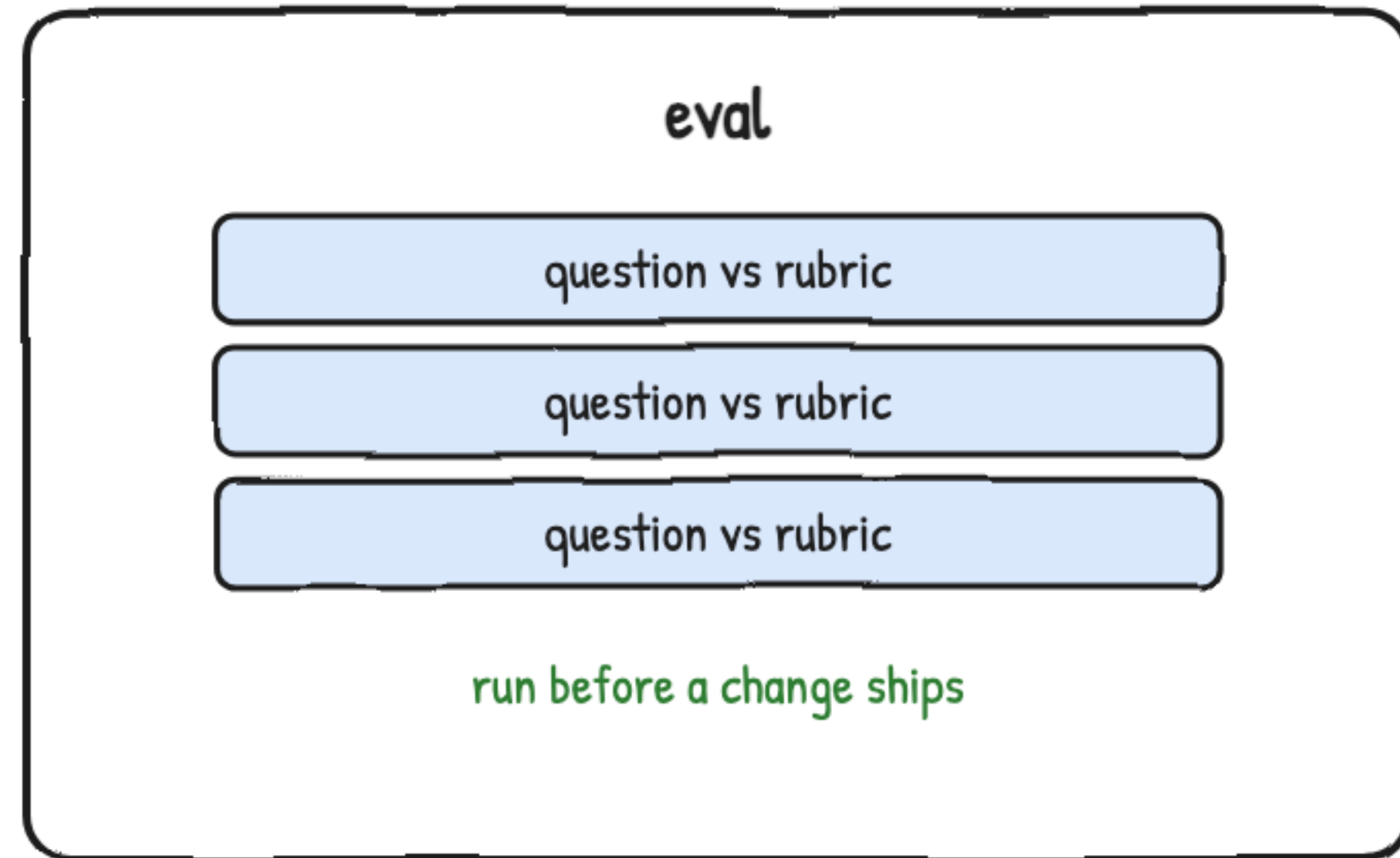
A structured request, not a direct answer

Tool-calling is the wire signal that lets an agent act



A structured request, not a direct answer

One checks before you ship, one blocks while it runs



Eval scores answers, guardrail stops actions

Hallucination is confident, fluent, and wrong

hallucination

"According to the 2019
Stanford study..." (invented)

stated with no hedge at all

honest answer

"I don't have that
information"

flags its own uncertainty

Not the same as an honest "I don't know"

The release notes from slide 3 decode themselves now

"context window
expanded to 200K tokens"

context window · tokens

"now supports
tool-calling"

agent · tool-calling

"hallucination rate
down 30% on our eval"

hallucination · eval

"70B parameters,
better answers"

parameters

you can read this now

ask which retrieval, over which docs?

Most pitches use four or five of these words, loosely

MODULE 2 TAKEAWAYS

You have the ten words — start the glossary



- 1 inside the model — tokens · parameters · embeddings · context window
- 2 its main failure — hallucination
- 3 layered on top — RAG
- 4 the agent loop — agent · tool-calling
- 5 watching both — eval · guardrail

Now open Module 2 · Lab. · Gourav Shah · School of DevOps & AI